

Guía Técnica de Arquitectura

Proyecto: VillaguayConectado

Documento interno de desarrollo

1. Objetivo de este documento

Este documento tiene como finalidad establecer una guía técnica para el desarrollo de VillaguayConectado antes de comenzar la etapa de programación.

Su objetivo es mantener una arquitectura consistente durante todo el proyecto, evitando improvisaciones y permitiendo que cada decisión técnica tenga un motivo claro.

Este documento funcionará como referencia durante el desarrollo y podrá actualizarse cuando el proyecto evolucione o aparezcan nuevas necesidades.

No reemplaza la documentación entregada a la cátedra, sino que complementa el proceso de desarrollo desde un punto de vista técnico.

2. Objetivo del proyecto

VillaguayConectado será una aplicación web destinada a centralizar la difusión de eventos locales de la ciudad de Villaguay.

La primera versión tendrá como único objetivo resolver correctamente el módulo de eventos.

No se desarrollarán durante esta etapa funcionalidades relacionadas con noticias, comercios, servicios, cupones, sistema de puntos o marketplace.

Estas funciones forman parte de la visión futura del proyecto, pero quedan completamente fuera del MVP.

Nuestro objetivo no es desarrollar muchas funcionalidades.

Nuestro objetivo es desarrollar pocas funcionalidades, pero correctamente.

3. Filosofía de desarrollo

Durante todo el proyecto se seguirán los siguientes principios.

Simplicidad

Siempre se elegirá la solución más simple posible, siempre que permita cumplir correctamente el objetivo.

No se agregarán funcionalidades únicamente porque "sería bueno tenerlas".

Escalabilidad

Aunque el MVP será pequeño, toda la arquitectura deberá permitir agregar nuevos módulos en el futuro sin tener que reescribir el proyecto.

Ejemplos:

- Noticias.
 - Comercios.
 - Servicios.
 - Sistema de puntos.
 - Beneficios.
 - Marketplace local.
-

Reutilización

Siempre que sea posible se crearán componentes reutilizables.

Ejemplos:

- Botones.
 - Inputs.
 - Cards.
 - Popups.
 - Formularios.
 - Modales.
-

Organización

Cada archivo deberá tener un único propósito.

No se crearán archivos enormes con muchas responsabilidades.

Código legible

Se priorizará siempre un código fácil de leer antes que un código excesivamente optimizado.

Este proyecto será evaluado por docentes y posiblemente retomado en el futuro.

4. Alcance del MVP

La primera versión deberá incluir únicamente las siguientes funcionalidades.

Público

- Consultar eventos.
- Buscar eventos.
- Filtrar eventos.
- Ver detalle de un evento.
- Abrir Google Maps.
- Compartir evento.
- Marcar "Me interesa".

Usuario registrado

Todo lo anterior más:

- Iniciar sesión.
- Registrarse.
- Proponer evento.
- Subir imagen.
- Esperar aprobación.

Administrador

Todo lo anterior más:

- Ver eventos pendientes.
- Aprobar evento.
- Rechazar evento.

5. Tecnologías del proyecto

Frontend

- React 19
- Vite
- React Router DOM
- CSS Modules

Backend

- Firebase Authentication
- Firebase Firestore
- Firebase Storage

Deploy

- Vercel

Control de versiones

- Git
- GitHub

Editor

- Visual Studio Code
- GitHub Copilot

6. Arquitectura general

El proyecto seguirá una arquitectura basada en componentes.

La interfaz estará completamente desacoplada de Firebase.

React únicamente solicitará datos mediante servicios.

Nunca accederá directamente a Firebase desde cualquier componente.

La comunicación será:

React

↓

Services

↓

Firebase

↓

Firestore / Storage / Authentication

Esto permitirá modificar la lógica del backend sin afectar la interfaz.

7. Arquitectura de carpetas

src/

|

└─ assets/

```
|
├─ components/
|   ├─ Button/
|   ├─ Card/
|   ├─ Navbar/
|   ├─ Footer/
|   ├─ Modal/
|   ├─ Input/
|   └─ Loader/
|
├─ layouts/
|
├─ pages/
|   ├─ Home/
|   ├─ Events/
|   ├─ EventDetail/
|   ├─ CreateEvent/
|   ├─ Community/
|   ├─ About/
|   ├─ Contact/
|   ├─ Login/
|   ├─ Register/
|   └─ Admin/
|
├─ hooks/
|
├─ contexts/
|
├─ firebase/
|
├─ services/
|
```

```
├─ types/
|
├─ utils/
|
├─ styles/
|
├─ App.jsx
└─ main.jsx
```

8. Arquitectura de Firebase

Se utilizarán únicamente tres servicios de Firebase.

Authentication

Responsable de:

- Registro.
 - Inicio de sesión.
 - Cierre de sesión.
 - Identificación del usuario.
 - Roles.
-

Firestore

Responsable de almacenar:

- Usuarios.
 - Eventos.
 - Estados.
 - Intereses.
-

Storage

Responsable únicamente del almacenamiento de imágenes.

Storage NO almacenará información del evento.

Solamente almacenará archivos.

Firestore almacenará la URL correspondiente.

9. Modelo de datos

users

id

displayName

email

photoURL

role

createdAt

Roles

admin

user

events

id

title

description

category

date

time

location

mapUrl

imageUrl

socialLinks

createdBy

status

createdAt

Estados

pending

approved

rejected

interests

id

eventId

userId

createdAt

10. Flujo de creación de un evento

Usuario

↓

Completa formulario

↓

Selecciona imagen

↓

Firebase Storage

↓

Obtiene URL

↓

Guardar documento

↓

Firestore

↓

status = pending

↓

Administrador revisa

↓

approved

↓

Evento visible para todos

11. Flujo de navegación

Inicio

↓

Eventos

↓

Detalle

↓

Google Maps

↓

Compartir

↓

Me interesa

Login

↓

Proponer evento

↓

Formulario

↓

Subir imagen

↓

Pendiente

↓

Administrador

↓

Aprobado

12. Convenciones de desarrollo

Carpetas

Siempre en minúsculas.

Ejemplo

components

services

utils

Componentes

Siempre en PascalCase.

EventCard.jsx

Navbar.jsx

Footer.jsx

Variables

Siempre en camelCase.

eventTitle

currentUser

selectedCategory

Colecciones Firestore

Siempre en inglés.

users

events

interests

Nunca utilizar nombres en español.

Rutas

Siempre en kebab-case.

/

eventos

proponer-evento

sobre

CSS

Cada componente tendrá su propio archivo.

Ejemplo

`EventCard.jsx`

`EventCard.module.css`

Nunca habrá un CSS gigante para toda la aplicación.

13. Componentes reutilizables previstos

Desde el inicio se intentará reutilizar componentes para evitar código duplicado.

Componentes previstos:

- Navbar
 - Footer
 - Button
 - Input
 - TextArea
 - Select
 - Card
 - EventCard
 - EventGrid
 - Badge
 - Modal
 - Popup
 - Loader
 - EmptyState
 - ProtectedRoute
-

14. Checklist del MVP

Base

- ☐ Crear proyecto React
- ☐ Configurar Firebase
- ☐ Configurar GitHub

☐ Configurar Vercel

Autenticación

- ☐ Registro
 - ☐ Login
 - ☐ Logout
 - ☐ Protección de rutas
-

Eventos

- ☐ Listado
 - ☐ Detalle
 - ☐ Filtros
 - ☐ Compartir
 - ☐ Me interesa
-

Propuesta

- ☐ Formulario
 - ☐ Validaciones
 - ☐ Subida de imagen
 - ☐ Guardado en Firestore
-

Administración

- ☐ Login administrador
 - ☐ Eventos pendientes
 - ☐ Aprobar
 - ☐ Rechazar
-

Diseño

- ☐ Responsive

☐ Accesibilidad

☐ CSS Modules

Deploy

☐ Producción

☐ README

☐ Revisión final

15. Decisiones técnicas adoptadas

A lo largo del desarrollo pueden surgir nuevas ideas. Sin embargo, antes de incorporar cualquier funcionalidad se deberá verificar que no contradiga el alcance definido para el MVP.

Las funcionalidades propuestas para futuras versiones (noticias, comercios, servicios, sistema de puntos, beneficios, marketplace u otras similares) no se implementarán durante esta primera etapa.

Toda nueva decisión deberá respetar la arquitectura definida en este documento y priorizar la estabilidad, la organización y la simplicidad del proyecto.

16. Observaciones finales

VillaguayConectado no busca ser únicamente un proyecto para aprobar una materia. Se plantea como una aplicación con potencial de crecimiento, por lo que las decisiones tomadas desde el inicio apuntan a construir una base sólida, organizada y escalable.

El objetivo principal será desarrollar una primera versión funcional, clara y mantenible, que permita demostrar los conocimientos adquiridos durante la carrera y, al mismo tiempo, sirva como punto de partida para futuras ampliaciones una vez finalizada la etapa académica.